

LESSON - 13



SIKHOGE KYA???

INTRODUCTION & KEY POINTS

- INTRODUCTION
- OVERVIEW OF DATABASES
- APPLICATIONS & JOBS
- SOFTWARE INSTALLATION -
- MYSQL SERVER AND MYSQL WORK BENCH WITH DATABASE SETUP
- TABLES CREATION & FIRST QUERY

SQL STATEMENT FUNDAMENTALS

- SELECT * FROM
- SELECT DISTINCT
- COUNT, ALIAS AS
- SELECT WHERE
- LIMIT AND ORDER BY
- OPERATORS AND FILTERING
- BETWEEN, IN, LIKE AND ILIKE
- CASE IF ELSE
- COALESCE, NULLIF



DATA MANIPULATION (DML)

- INSERT
- UPDATE
- DELETE
- COPY RENAME STRUCTURE
- CHECK CONSTRAINT

DATA DEFINITION LANGUAGE (DDL)

- DATA TYPES & ROWS COLUMNS
- PRIMARY FOREIGN & OTHER KEYS
- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CONSTRAINTS

JOINS & SUB QUERY

- INNER JOIN
- LEFT JOIN (LEFT OUTER JOIN)
- RIGHT JOIN (RIGHT OUTER JOIN)
- FULL JOIN (FULL OUTER JOIN)
- CROSS JOIN
- SELF JOIN
- -----
- UNION, UNION ALL, MINUS, INTERSECT
- STRING FUNCTIONS AND OPERATORS
- SUBQUERY - IMPORTANT
- DATE & TIME FUNCTIONS

GROUP BY & HAVING

- INTRODUCTION TO GROUP BY
- AGGREGATION FUNCTIONS
- GROUP BY - CHALLENGE
- HAVING
- HAVING - CHALLENGE TASKS

ADVANCE SQL

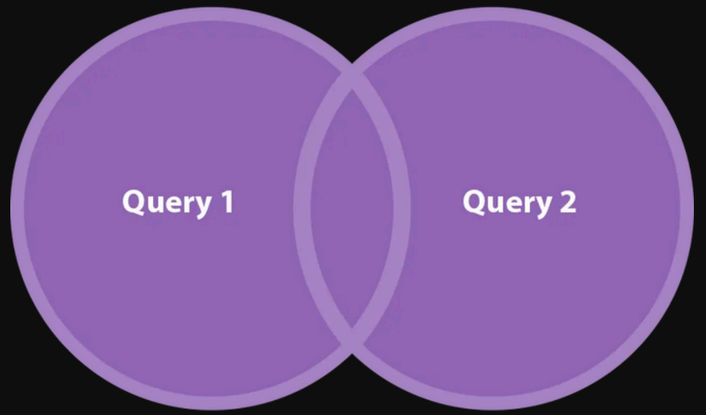
- VIEWS / TEMP TABLES & INDEXES
- FUNCTIONS & TRIGGERS
- STORE PROCEDURES
- TRANSACTIONS
- WINDOW FUNCTIONS
- QUERY ANALYSIS TECHNIQUES

UNION

UNION merges the results of two SELECT statements. Important: UNION statements only return UNIQUE values.

Syntax:

```
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2;
```



Rules

- Each SELECT statement within the UNION must have the same number of columns.
- Data Types Must Be Compatible
- Removes Duplicates by Default (UNION vs UNION ALL)
- Order of Results : If you want to order the results, use ORDER BY only once at the end of the final query.
- You can add JOINs & Where conditions in both tables or single tables too.

UNION ALL

Combines the results of two SELECT queries but includes all duplicates.
Faster than UNION because it doesn't check for duplicates.

Syntax:

```
SELECT column1, column2 FROM table1
UNION ALL
SELECT column1, column2 FROM table2;
```

Example:

```
SELECT name FROM customers
 UNION ALL
SELECT name FROM suppliers;
```



MINUS SET

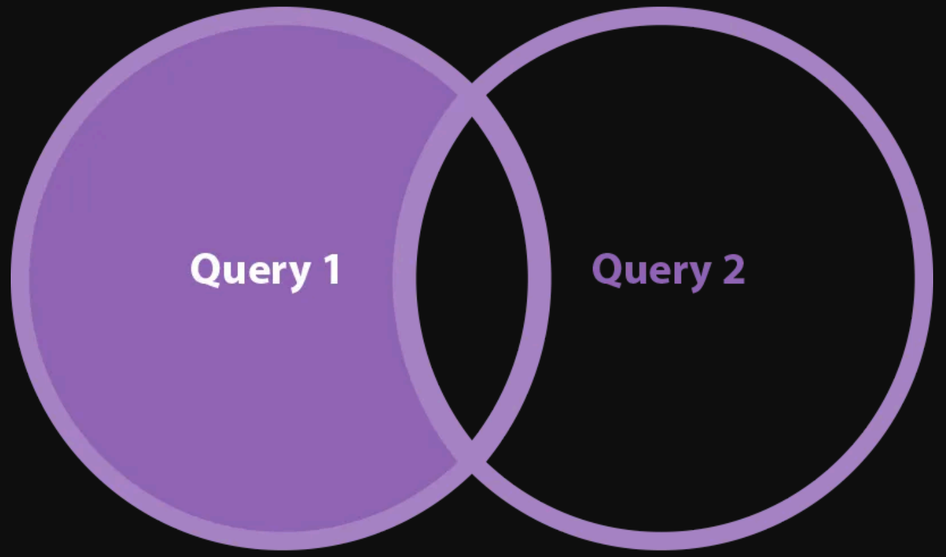
Returns rows from the first SELECT statement that are not present in the second SELECT statement.

Syntax:

```
SELECT column1, column2 FROM table1
MINUS
SELECT column1, column2 FROM table2;
```

Example:

```
SELECT name FROM customers
 MINUS
SELECT name FROM suppliers;
```



INTERSECT SET

Returns only the common rows from both SELECT statements.
Removes duplicates automatically.

Syntax:

```
SELECT column1, column2 FROM table1
INTERSECT
SELECT column1, column2 FROM table2;
```

Example:

```
SELECT name FROM customers
INTERSECT
SELECT name FROM suppliers;
```

